
ACOS: Governed Execution for Agents in the Wild

Policy Gateway, Runtime Executor, and MCP Behind the Boundary

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Tool-using agents inherit backend privilege: a single prompt injection or salami-
2 sliced trajectory can turn probabilistic text into irreversible side effects. We
3 present **ACOS** (Aegis Cortex OS), a governed execution boundary that separates
4 *intent proposal* from *physical execution*. A deterministic **Policy Gateway** main-
5 tains cross-step risk state and emits explicit APPROVED/REJECTED decisions; a
6 **Runtime Executor** is the sole component allowed to invoke tool backends—
7 including MCP—and only after approval. We state a concrete threat model and L1
8 guarantees, make Risk Engine profiles and fixture signals reproducible, compare
9 stateful vs. stateless vs. hard-guard baselines on salami and public-bench-proxy
10 trajectories, and report sensitivity ablations. Beyond mocks, we ship a read-only
11 filesystem connector and an MCP-behind-Executor sketch with honest L1 lim-
12 its. ACOS complements orchestrators (e.g., LangGraph): it authorizes execution
13 rather than routing workflows.

14 1 Introduction

15 Most agent stacks still follow

user → LLM → tool call → API/DB/FS.

16 Security is then layered on with prompts, filters, or post-hoc review. That design fails under prompt
17 injection, gradual privilege escalation (“salami-slicing”), runaway loops, and audit ambiguity be-
18 tween “model refused” and “policy blocked.”

19 **ACOS** enforces *proposal* ≠ *execution*:

- 20 1. **Intent Proposer** (LLM) emits structured intents only (tool_call, final_answer,
21 terminate).
- 22 2. **Policy Gateway** arbitrates with ingress budgets, whitelist / schema guards, and a stateful
23 Risk Engine.
- 24 3. **Runtime Executor** dispatches to a tool registry *only* on APPROVED.

25 Tool transports such as the Model Context Protocol (MCP) [Anthropic and community, 2024] be-
26 long *behind* the Executor, not beside the model. Positioning: *LangGraph orchestrates; Aegis autho-*
27 *rizes* [LangChain, 2024, ACOS Authors, 2026].

28 This workshop revision addresses reviewer and OpenReview-style requests for related-work place-
29 ment, an explicit threat model, reproducible Risk Engine definitions, empirical baselines (includ-
30 ing public-bench proxies), beyond-mock connectors, and sensitivity analysis (revision TODOs 1–7;
31 checklist in the companion artifact).

32 2 Related Work

33 **Dual LLM/ CaMeL and privilege separation.** Dual-LLM and CaMeL-style designs reduce blast
34 radius by separating privileged and unprivileged model contexts, or by constraining how untrusted
35 content flows into tool arguments [Security community, 2024]. ACOS is complementary: it does
36 not claim to solve prompt understanding; it places a deterministic authorize/execute cut after intent
37 formation. Even a dual-LLM proposal must still face an execution boundary if tools can cause
38 irreversible effects. Where Dual LLM/CaMeL primarily reshape *information flow into the model*,
39 ACOS reshapes *authority flow out of the model*.

40 **Policy gateways, sandboxes, and orchestrators.** IAM gateways, syscall sandboxes,
41 WASM/containers, and agent orchestrators (e.g., LangGraph [LangChain, 2024]) and MCP
42 tool hosts [Anthropic and community, 2024] each address part of the problem. Orchestrators excel
43 at routing and state machines; OS sandboxes constrain process capability; many “guardrails” score
44 a single step. ACOS’s novelty is the combination of (i) an explicit proposal/gateway/executor
45 *execution boundary*, (ii) *cross-step* risk state that catches compositional escalation, and (iii) *MCP*
46 *(and other backends) placed behind the Executor* so adding servers does not automatically enlarge
47 model privilege.

48 **State overlap vs. novelty.** Static allowlists, schema validators, and per-step criticality scores over-
49 lap with ACOS hard guards—intentionally. The Risk Engine targets graded, temporal threats those
50 guards miss: each step may look locally safe while session risk accumulates. We do not claim
51 uniqueness for fail-closed dispatch alone; the contribution is the governed boundary plus repro-
52 ducible cross-step arbitration under an explicit L1 threat model.

53 3 Threat Model

54 **Attacker capabilities (in scope).** We assume an adversary who can (a) inject or manipulate
55 natural-language content observed by the proposer (prompt injection, untrusted tools/docs), (b) steer
56 multi-step trajectories where each individual tool call looks locally plausible (salami-slicing), and
57 (c) attempt data exfiltration or high-criticality actions (export, refund, external send) via registered
58 tools. The attacker does *not* need to break cryptography; success is inducing an APPROVED physical
59 side effect that violates policy.

60 **Out of scope.** Kernel exploits, stolen host credentials outside the agent path, supply-chain com-
61 promise of the Gateway binary, and covert channels that bypass the registry entirely are out of scope
62 for L1. Physical process/network isolation is an L2/L3 deployment concern (Section 8).

63 **L1 guarantees.** Given a correct host integration (no unguarded tool path):

- 64 • No registry handler runs unless the Gateway returns APPROVED (fail-closed Executor).
- 65 • Egress arbitration is deterministic given (D_t, E_t, C_T) and session risk state (replayable).
- 66 • Cross-step accumulation can veto compositional salami trajectories that per-step kinetic
67 checks miss (Section 6).
- 68 • Denies expose structured reasons (`physical_gate_status`, `termination_cause`) dis-
69 tinct from model text.

70 **Non-guarantees (honest).** L1 does *not* guarantee: OS-level confinement of a compromised pro-
71 poser process; calibrated probabilities from hand-chosen $(\kappa, \gamma, \tau, R_{\max})$; completeness against
72 novel injection that never proposes a tool call; or official Attack Success Rate on Agent-
73 Dojo/InjecAgent suites (we report policy-layer proxies only; Section 6.2).

74 4 System Design

75 **Ingress.** On each user request the Gateway issues a session ID, allocates token/step budget, runs
76 an ingress threat seed, and initializes a tool whitelist (empty if hijack is flagged).

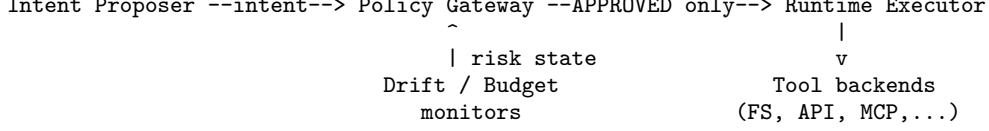


Figure 1: ACOS L1 boundary. MCP and other backends sit behind the Executor.

77 **Per-step sandbox.** Before each proposer step the reference runtime may re-prune the reachable
 78 tool surface using phase locks and drift signals. Whitelists are not necessarily static for the session
 79 lifetime.

80 **Egress.** Hard guards (budget, hijack flag, schema, whitelist) run first, then the Risk Engine (Sec-
 81 tion 5). Decisions: APPROVED, REJECTED, HARD_MELTDOWN. OVERRIDE is schema-reserved and
 82 non-auto-executable.

83 **Executor.** `execute_approved` re-validates payloads, looks up `PhysicalToolRegistry`, in-
 84 vokes handlers, and audits outcomes. Non-approved statuses never call tools [ACOS Authors, 2026].

85 **MCP placement.** Correct: `Intent` $\rightarrow$ `Gateway` $\rightarrow$ `Executor` $\rightarrow$ registry handler $\rightarrow$ MCP client.
 86 Incorrect: LLM/orchestrator direct to MCP; Gateway must not open MCP sessions. Integrators
 87 register each MCP tool (or family) as a handler with `criticality_score`; the Gateway sees only
 88 structured intents.

89 **Integration surface.** Path A (recommended): host orchestrators call `ingress_gate` /
 90 `egress_gate` / `execute_approved`. Path B: the reference loop `run_agent_os_once()` pack-
 91 ages the full contract for demos. LangGraph stock `ToolNode` paths that skip the Gateway defeat the
 92 model.

93 5 Risk Engine

94 Normalized inputs $D_t, E_t, C_T \in [0, 1]$:

- 95 • D_t : goal-drift score (monitor-derived),
- 96 • E_t : logical-entropy / instability score,
- 97 • C_T : tool criticality (read $\ll$ export/act).

98 Profiles $(\kappa, \gamma, \tau, R_{\max})$ are **hand-chosen heuristics**, not fitted estimators. Named profiles in the
 99 reference benchmark:

Profile	κ	γ	τ	$R_{\max}$
strict	4.0	0.8	1.5	1.0
balanced	2.0	0.8	1.5	1.5
research	1.5	0.75	1.2	2.5

101 Equations (reproduced in Appendix A):

$$\begin{aligned}
 R_{\text{kinetic}}(t) &= C_T(\exp(\min(\kappa D_t E_t, 10)) - 1), \\
 R_{\text{potential}}(t) &= \gamma R_{\text{potential}}(t-1) + R_{\text{kinetic}}(t), \\
 \Delta R(t) &= \max(0, R_{\text{kinetic}}(t) - R_{\text{kinetic}}(t-1)), \\
 R_{\text{eff}}(t) &= R_{\text{potential}}(t) + \tau \Delta R(t).
 \end{aligned}$$

102 Veto if $R_{\text{eff}}(t) \geq R_{\max}$. Reproduction: `auto_test/test_salami_slicing_benchmark.py`,
 103 `test_risk_policy_baselines.py`.

Table 1: Fixture suite FP/FN summary (balanced). Attack catch counts over two salami trajectories; FN = 1 − catch.

Policy	Attack catch	FN	Benign FP
stateful	2/2	0.00	0/2
stateless_kinetic	1/2	0.50	0/2
hardguard_only	0/2	1.00	0/2

Table 2: Public-bench *proxy* catch rates (not official ASR).

Policy	Attack catch	FN	Benign FP
stateful	4/4	0.00	0/2
stateless_kinetic	2/4	0.50	0/2
hardguard_only	0/4	1.00	0/2

6 Empirics

6.1 Baselines on salami fixtures

Policies: **stateful** (full equation); **stateless_kinetic** ($R_{\text{eff}} = R_{\text{kinetic}}$ only); **hardguard_only** (always approve at the risk layer—allowlist/schema without graded risk). Trajectories: **salami_original** (HR→VP→CEO→external email), **salami_compositional** (moderate final spike so kinetic-only may miss), plus two benign reads. Under balanced, Table 1 shows stateful 2/2 attack catch, stateless 1/2 (misses compositional), hardguard 0/2; benign FP = 0 for risk-aware policies. On **salami_original**, balanced vetoes at step 4; **strict** at step 2.

6.2 Public-bench proxy (AgentDojo / InjecAgent themes)

We evaluate *policy-layer* trajectories inspired by AgentDojo [Debenedetti et al., 2024] and InjecAgent [Zhan et al., 2024] themes (injection→escalate→exfil / tool misuse), including compositional finals. **Disclaimer:** this is **not** official suite ASR; we do not run the published harnesses. Historically the AgentDojo Python package required Python ≥ 3.10 ; our proxy uses stdlib unittest without that dependency. Signals (D_t, E_t, C_T) are hand-labeled to mirror attack shapes—not scraped from benchmark logs. On balanced, stateful catches 4/4 attacks with 0/2 benign FP; stateless 2/4; hardguard 0/4 (auto_test/test_public_bench_proxy.py).

6.3 Sensitivity

Sensitivity and ablations. Holding the fixture suite fixed, we vary one Risk Engine parameter at a time around balanced ($\kappa, \gamma, \tau, R_{\text{MAX}} = (2.0, 0.8, 1.5, 1.5)$) and report stateful attack catch / FN, benign FP, and veto step on **salami_original** / **salami_compositional**. Reproduce with python3 auto_test/test_risk_sensitivity.py.

Table 3: Named ablations (stateful policy on the fixture suite).

Setting	Catch	FN	Benign FP	Veto (orig / comp)
balanced	2/2	0.00	0/2	4 / 4
no memory ($\gamma=0$)	2/2	0.00	0/2	4 / 4
no momentum ($\tau=0$)	1/2	0.50	0/2	4 / —
kinetic only ($\gamma=\tau=0$)	1/2	0.50	0/2	4 / —
strict-like	2/2	0.00	0/2	2 / 2
research-like	1/2	0.50	0/2	4 / —

Takeaway. On these fixtures, compositional catch is most sensitive to τ and R_{MAX} (and to lowering κ); $\gamma=0$ alone still vetoes because $\tau \cdot \Delta R$ on the final spike remains large. Benign FP stays 0 near balanced. Strict-like thresholds veto earlier (step 2) while still counting as attack prevention. Profiles remain hand-chosen heuristics, not fitted estimators.

Table 4: One-at-a-time sweeps (others fixed at balanced).

Param	Values → catch	Comp. veto lost when	Benign FP
κ	1.0–1.5: 1/2; ≥ 2.0 : 2/2	$\kappa < 2$	0
γ	all tested: 2/2	(none in this grid)	0
τ	< 1.0 : 1/2; ≥ 1.0 : 2/2	$\tau < 1$	0
R_{MAX}	≤ 2.0 : 2/2; 2.5: 1/2	$R_{\text{MAX}} = 2.5$	0

7 Beyond Mocks

Read-only filesystem connector. `core_runtime/connectors/readonly_fs.py` resolves paths under an explicit root, rejects `..` / absolute escape, forbids writes, and exposes async handlers for registry registration. Unit tests verify `execute_approved` is required for I/O (`auto_test/test_connectors_realism.py`).

MCP-behind-Executor sketch. `core_runtime/connectors/mcp_backend.py` and `examples/sandbox_mcp_behind_executor.py` demonstrate the correct wiring: Gateway never speaks MCP; Executor-dispatched proxies call an in-process MCP-shaped catalog. This is an L1 sketch (no full JSON-RPC/stdio transport by default, no L2 isolation). Compromised in-process proposers remain an L1 limit; L2+ adds proxy/mount/syscall controls (Section 8).

What “beyond mocks” does and does not claim. We claim real filesystem reads under a chroot-like root and a faithful *placement* sketch for MCP. We do *not* claim production MCP interoperability, audited connectors, or enclave isolation.

8 Limitations and Conformance

Conformance levels: **L0** screening only; **L1** (this repo) Proposer/Gateway/Executor boundary with deterministic egress; **L2** infrastructure bypass controls; **L3** memory/process isolation. Known L1 gaps include physical isolation, unfinished `VERRIDE/ICU` behavior, regex ingress limits, and demo-grade connectors. Treat ACOS as a reference architecture and evaluation harness, not a certified appliance. No third-party security audit is claimed.

9 Conclusion

ACOS argues that agents in the wild need an authorize/execute boundary with cross-step risk and backends (including MCP) behind the Executor. We answered review concerns with an explicit threat model, reproducible equations and profiles, baseline/proxy empirics, sensitivity tables, and honest beyond-mock connectors. Future work: L2 isolation, calibrated risk estimators, and official public-bench harness integration under the same Gateway cut.

References

- ACOS Authors. ACOS: Aegis cortex OS — governed execution for AI agents, 2026. Reference implementation and whitepaper.
- Anthropic and community. Model context protocol, 2024. Available at <https://modelcontextprotocol.io>.
- Edoardo DeBenedetti et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *NeurIPS Datasets and Benchmarks*, 2024.
- LangChain. LangGraph, 2024. Available at <https://langchain-ai.github.io/langgraph/>.
- Security community. CaMeL / dual-LLM privilege separation for tool-using agents, 2024. Dual-LLM and capability-mediated designs; cited for positioning.
- Qiusi Zhan et al. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. *arXiv preprint*, 2024.

166 A Risk equation details

167 **Full recurrence.** Let $R_{\text{potential}}(0) = 0$ and $R_{\text{kinetic}}(0) = 0$. For each step $t \geq 1$ with signals
 168 (D_t, E_t, C_T) :

$$R_{\text{kinetic}}(t) = C_T(\exp(\min(\kappa D_t E_t, 10)) - 1), \quad (1)$$

$$R_{\text{potential}}(t) = \gamma R_{\text{potential}}(t-1) + R_{\text{kinetic}}(t), \quad (2)$$

$$\Delta R(t) = \max(0, R_{\text{kinetic}}(t) - R_{\text{kinetic}}(t-1)), \quad (3)$$

$$R_{\text{eff}}(t) = R_{\text{potential}}(t) + \tau \Delta R(t). \quad (4)$$

169 Decision: APPROVED iff $R_{\text{eff}}(t) < R_{\text{max}}$ after hard guards pass; else REJECTED (or HARD_MELTDOWN
 170 for metabolic trips).

171 **Numerical safety.** Exponent inputs are clamped with $\min(\kappa D_t E_t, 10)$. Non-finite arithmetic
 172 must fail closed. Hard guards precede the equation (step budget, hijack, schema, whitelist).

173 **Signal definitions (operational).** In the reference runtime, D_t and E_t are pro-
 174 duced by drift/entropy monitors over reasoning and payload features; C_T comes from
 175 ToolSpec.criticality_score. Fixture benchmarks inject explicit scalars so arbitration is
 176 bit-reproducible without LLM sampling.

177 **Salami fixture signals (balanced veto at step 4).** salami_original: (D_t, E_t, C_T) sequence
 178 $(0.05, 0.10, 0.50)$, $(0.60, 0.30, 0.50)$, $(0.65, 0.30, 0.50)$, $(0.90, 0.80, 0.90)$. Compositional replaces
 179 the final spike with $(0.70, 0.55, 0.70)$ so kinetic-only stays under R_{max} while stateful crosses it.

180 Reproduction commands.

```
181 python3 auto_test/test_salami_slicing_benchmark.py
182 python3 auto_test/test_risk_policy_baselines.py
183 python3 auto_test/test_risk_sensitivity.py
184 python3 auto_test/test_public_bench_proxy.py
185 python3 -m unittest auto_test.test_connectors_realism -v
186 python3 examples/sandbox_mcp_behind_executor.py
```

187 B Extended sensitivity notes

188 See also docs/risk_sensitivity.md. Sweep highlights on the fixture suite: $\kappa < 2$ or $\tau < 1$ or
 189 $R_{\text{max}} = 2.5$ can miss compositional; $\gamma = 0$ alone still vetoes via $\tau \Delta R$ on the final spike; benign
 190 FP stays 0 near balanced for $\kappa \in \{1.5, 2, 3\}$. Strict-like thresholds veto earlier (step 2) while still
 191 counting as attack prevention. Profiles remain hand-chosen heuristics, not fitted estimators.